

polars-bio w trybie rozproszonym: Ballista i Sail

Wy tłumaczenie krok po kroku — od podstaw

Miłosz Kowalewski

26 sierpnia 2026

Spis treści

Wstęp — o co w ogóle chodzi	1
Rozdział 1 — Cegielki, na których to wszystko stoi	2
Apache Arrow — wspólny język danych	2
DataFusion — silnik, który wykonuje zapytania	2
polars-bio i COITrees — sedno algorytmu	2
Rozdział 2 — Ballista: rozproszenie „z tej samej rodziny”	2
Problem: jak przesłać niestandardowy „przepis”?	3
Rozwiązanie: własny „tłumacz” (codec)	3
Dogonienie: COITrees też w wersji rozproszonej	3
Rozdział 3 — Sail: inna rodzina, inna droga	4
Jak to działa	4
Problemy napotkane po drodze	5
Eksperyment: czy Sail da się uruchomić NAPRAWDĘ rozproszony?	5
Jak to sprawdzono	5
Wniosek	5
Co to jest Kubernetes i dlaczego to komplikuje sprawę	6
Co to jest taki „limit pamięci” (cgroup)	6
Co poszło nie tak	6
Co się stało dalej	6
Co z tego wynika	7
Rozdział 4 — Co to wszystko znaczy dla całej pracy magisterskiej	7

Wstęp — o co w ogóle chodzi

Wyobraź sobie, że masz ogromną książkę telefoniczną — tak dużą, że nie mieści się w pamięci jednego komputera. Chcesz w niej znaleźć wszystkie pary osób, które mieszkają na tej samej ulicy. Jeden komputer by się z tym męczył (albo w ogóle by się nie dało, bo dane nie mieszczą się w RAM-ie). Rozwiązanie: pociąć książkę na kawałki, rozdać je wielu komputerom naraz, każdy przeszuka swój kawałek, a na końcu ktoś skleja wyniki. To właśnie nazywa się **przetwarzaniem rozproszonym**.

W tej pracy magisterskiej „książką telefoniczną” są dane genomiczne — np. miliony odcinków DNA opisanych jako (chromosom, początek, koniec). Operacja, o którą chodzi najczęściej, to **overlap**: „znajdź wszystkie pary odcinków, które się nakładają”. Jest jeszcze kilka innych

podobnych operacji (merge — sklej nakładające się odcinki w jeden; nearest — znajdź najbliższy sąsiedni odcinek; coverage — ile razy dany fragment jest pokryty; subtract — odejmij jedne odcinki od drugich).

Do liczenia tych operacji na jednym komputerze służy biblioteka **polars-bio** (współautor: promotor tej pracy). Jest bardzo szybka, ale działa tylko na jednej maszynie. Cel tej pracy: sprawdzić, czy da się polars-bio „dowieźć” do dwóch popularnych silników do przetwarzania rozproszonego — **Apache Ballista** i **Sail** — tak żeby te same obliczenia dało się rozłożyć na wiele komputerów, bez przepisywania polars-bio od zera i bez „forkowania” (kopiowania i modyfikowania na stałe) tych silników.

Rozdział 1 — Cegielki, na których to wszystko stoi

Apache Arrow — wspólny język danych

Różne programy (Python, Rust, Java) zwykle trzymają dane w pamięci na swój własny sposób, więc żeby jeden program przekazał dane drugiemu, musi je „przetłumaczyć” — co jest wolne. **Apache Arrow** to uzgodniony, wspólny format przechowywania danych tabelarycznych w pamięci, który rozumieją wszystkie zaangażowane tu technologie (polars-bio, DataFusion, Ballista, Sail). Dzięki temu nie trzeba niczego tłumaczyć — to jak gdyby wszyscy w firmie nagle zaczęli mówić tym samym językiem, zamiast przez tłumacza.

DataFusion — silnik, który wykonuje zapytania

DataFusion to biblioteka (napisana w języku Rust), która potrafi wziąć zapytanie w stylu SQL („wybierz te wiersze, połącz te dwie tabele, policz sumę”) i wykonać je szybko na danych w formacie Arrow. Nie jest to gotowy program z interfejsem — to raczej „silnik pod maską”, którego używają inne programy. Zarówno polars-bio, jak i Ballista, jak i Sail — wszystkie trzy są zbudowane na DataFusion (Sail nieco inaczej, o czym niżej). Dzięki temu współdzielą fundament: sposób opisywania planu zapytania (czyli „co dokładnie trzeba policzyć i w jakiej kolejności”).

polars-bio i COITrees — sedno algorytmu

polars-bio to biblioteka w Pythonie do analiz genomicznych, zbudowana na DataFusion. Kiedy wywołasz `pb.overlap(a, b)`, tak naprawdę pod spodem woła ona zewnętrzny komponent — `datafusion-bio-function-ranges` — który implementuje prawdziwy algorytm. Jednym z dostępnych algorytmów jest **COITrees** — sprytna struktura danych (drzewo przedziałów), która pozwala sprawdzić „czy ten odcinek nakłada się z czymś” dużo szybciej niż porównywanie każdego z każdym (to ostatnie dla dużych danych byłoby beznadziejnie wolne — jak sprawdzanie wszystkich par osób w mieście, zamiast najpierw pogrupować je ulicami).

Ważne odkrycie na starcie tej pracy: pierwszy prototyp w ogóle nie używał tego prawdziwego silnika — liczył nakładanie się odcinków „na piechotę” (porównanie każdy z każdym). Jednym z pierwszych kroków było więc podpięcie prawdziwego silnika polars-bio, żeby porównanie z silnikami rozproszonymi było uczciwe.

Rozdział 2 — Ballista: rozproszenie „z tej samej rodziny”

Apache Ballista to nakładka na DataFusion, która potrafi rozdzielić zapytanie na wiele komputerów. Ma dwie główne role:

```

KLIENT (Twój program)
| wysyła plan zapytania
v
SCHEDULER ---- rozdziela zadania --> EXECUTOR 1
|                                     EXECUTOR 2
|<----- zbiera wyniki ----- EXECUTOR 3
v
wynik z powrotem do klienta

```

Scheduler to „dyrygent” — dostaje zapytanie, dzieli dane na kawałki (np. po chromosomie: „chr1” do jednego workera, „chr2” do drugiego) i rozsyła zadania. **Executory** to „robotnicy” — każdy dostaje swój kawałek danych i swoją część zadania, liczy ją lokalnie, i odsyła wynik z powrotem. Ruch danych między schedulerem a executorami nazywa się **shuffle**.

Ponieważ scheduler i executor to osobne procesy (czasem nawet osobne komputery), trzeba im przesłać nie tylko dane, ale też sam **plan zapytania** — czyli dokładny „przepis”, co i jak policzyć. Zwykle, standardowe operacje (jak „posortuj” czy „policz sumę”) DataFusion potrafi zamienić na taki przesyłalny przepis samodzielnie. Problem pojawia się, gdy chcemy użyć czegoś niestandardowego — jak właśnie nasz operator overlap z polars-bio.

Problem: jak przesłać niestandardowy „przepis”?

Kiedy spróbowano po raz pierwszy wysłać zapytanie z overlap do prawdziwego klastra Ballista, dostano błąd: *„LogicalExtensionCodec is not provided”* — po polsku: „nie wiem, jak zapisać ten kawałek przepisu do przesłania, bo to coś niestandardowego, nie mam do tego instrukcji tłumaczenia”. To jak wysłanie komuś przepisu kulinarnego z jednym krokiem napisanym w nieznanym języku — reszta przepisu jest zrozumiała, ale tego jednego kroku odbiorca nie umie odczytać.

Rozwiązanie: własny „tłumacz” (codec)

Napisano własny **kodek** (LogicalExtensionCodec) — czyli dokładnie taką instrukcję tłumaczenia dla tego jednego niestandardowego kroku: „jeśli zobaczysz krok o nazwie X, to zapisz go jako te trzy liczby/napisy, a po drugiej stronie odtwórz go z powrotem z tych trzech liczb/napisów”. Nie trzeba było przy tym w żaden sposób modyfikować samej Ballisty — to mechanizm rozszerzeń, który Ballista udostępnia z założenia, dokładnie po to, żeby dało się do niej dopisywać nowe rzeczy bez grzebania w jej wnętrzu.

Efekt pierwszej udanej próby: zapytanie z overlap policzone poprawnie na prawdziwym klastrze (scheduler + kilku executorów, prawdziwy shuffle danych między nimi) — ale z jednym kompromisem: executor liczył wynik zwykłym, generycznym sposobem łączenia tabel (HashJoinExec), a nie sprytnym COITrees. Czyli rozproszenie działało, ale bez najszybszego algorytmu w środku.

Dogonienie: COITrees też w wersji rozproszonej

W tej sesji postawiono sobie za cel, żeby to naprawić — użyć COITrees **także** w wersji w pełni rozproszonej, nie tylko lokalnie na jednym komputerze.

Sedno problemu: sam mechanizm liczący przez COITrees (IntervalJoinExec) był dostępny „z zewnątrz” prawie w całości — poza jednym drobnym elementem: strukturą opisującą, które kolumny to początek/koniec odcinka (ColIntervals), która była ukryta „za ścianą” — schowana w module, do którego kod spoza tej biblioteki nie ma dostępu (to trochę jak szuflada w czyimś biurku, zamknięta na klucz, mimo że rzecz w środku wcale nie jest tajna — po prostu nikt nie zostawił do niej klucza dla osób z zewnątrz).

Rozwiązanie: zamiast rezygnować, zrobiono lokalną kopię tej jednej, pomocniczej biblioteki (**nie** Ballisty ani Saila — tylko małego komponentu, który dostarcza sam algorytm) i zmieniono w niej dosłownie jedno słowo w kodzie źródłowym: `mod` na `pub mod` (czyli „ten moduł jest prywatny” → „ten moduł jest publiczny”) — odpowiednik zostawienia klucza do tej szuflady również dla gości. To udokumentowana, przejrzysta, minimalna zmiana — nie fork silnika rozproszonego, tylko drobna poprawka widoczności w bibliotece pomocniczej, gotowa do zaproponowania z powrotem autorom oryginału.

Po tej poprawce udało się napisać drugi, bardziej zaawansowany kodek — tym razem dla **fizycznego** planu (`PhysicalExtensionCodec`), czyli przepisu na poziomie „executor do executora”, a nie tylko „klient do schedulera”. Po drodze napotkano jeszcze jeden błąd uruchomieniowy (`PartitionMode Auto` — coś w rodzaju „nie zdążono ustalić, w jaki sposób dane mają zostać podzielone między executy, zanim faktycznie zaczęto liczyć” — naprawione przez jawne wymuszenie konkretnego trybu podziału).

Wynik końcowy Części 1: to samo zapytanie `overlap`, na prawdziwym, rozproszonym klastrze Ballista, tym razem faktycznie liczone przez COITrees na każdym executorze — potwierdzone identycznym wynikiem jak przy liczeniu na jednym komputerze. To jest największe osiągnięcie tej sesji: pełna dystrybucja **bez** utraty szybkiego algorytmu.

Rozdział 3 — Sail: inna rodzina, inna droga

Sail to inny silnik rozproszony — zamiast być bezpośrednio zbudowany na tych samych mechanizmach co Ballista, mówi tzw. protokołem **Spark Connect** — czyli protokołem, którym komunikuje się (Py)Spark, bardzo popularne narzędzie do przetwarzania rozproszonego. Dzięki temu programy pisane pod PySparka mogą łączyć się z Sailem, nie wiedząc nawet, że to nie jest prawdziwy Spark.

Ponieważ Sail nie należy do „rodziny DataFusion” w takim samym sensie jak Ballista, nie da się mu bezpośrednio wstrzyknąć niestandardowego kroku planu zapytania tak jak zrobiono to z kodekami w Ballistrze (mechanizm do tego, nazwany roboczo `SailExtension`, jest dopiero projektowany przez zespół Saila — nie jest gotowy). Zamiast tego użyto innego, w pełni legalnego mechanizmu: **UDTF** (*User-Defined Table Function* — funkcja zdefiniowana przez użytkownika, która na wejściu i wyjściu operuje na całych tabelach, nie pojedynczych wartościach).

Jak to działa

Sail dzieli dane na grupy (po chromosomie) -- standardowa operacja, jak `"group by"`

|

v

Dla każdej grupy: Sail woła funkcję UDTF, którą sami zarejestrowaliśmy

|

v

Wewnątrz UDTF: nasz kod woła z powrotem `pb.overlap()` -- prawdziwe, niezmienione `polars-bio`

|

v

Wynik wraca do Saila, Sail składa wszystko w jedną tabelę

Innymi słowy: Sail robi to, co umie najlepiej (dzielenie i rozsyłanie pracy), a właściwe liczenie `overlap` dalej robi `polars-bio` — tylko wywoływane osobno na każdej porcji danych.

Problemy napotkane po drodze

1. **UDTF nie przyjmuje całych tabel jako argumentu** — w wersji Saila użytej w tej pracy dało się przekazać do UDTF tylko pojedyncze wartości (np. nazwę chromosomu, listę odcinków jako jedną skomplikowaną wartość), a nie „prawdziwą” tabelę z wieloma wierszami jako osobny argument. Trzeba było to obejść, pakując dane w listy przekazywane jako pojedyncza wartość.
2. **Błąd przy wielu partycjach jednocześnie** — kiedy dane były podzielone na więcej niż jeden kawałek i UDTF uruchamiał się na kilku kawałkach naraz, część wyników znikła, a część się duplikowała. To błąd samego Saila (a nie naszego kodu) w sposobie łączenia wyników z wieloma kawałkami przy tego typu wywołaniu. Obejście: wymuszenie jednego kawałka danych przed wywołaniem UDTF — naprawia poprawność wyniku, ale kosztem tego, że na tym etapie nie widać jeszcze przyspieszenia z posiadania wielu komputerów (to do poprawy/zbadania w przyszłości).
3. **Rejestracja funkcji „w złym momencie”** — sposób, w jaki Sail sprawdza czy program działa w trybie lokalnym czy rozproszonym, robi to zbyt wcześnie (w momencie pisania kodu funkcji, a nie w momencie jej faktycznego zarejestrowania) — trzeba było zmienić sposób organizacji kodu, żeby to zadziałało poprawnie.

Wynik: ta sama operacja overlap, wywołana przez Saila, dająca identyczny wynik jak lokalne polars-bio.

Eksperyment: czy Sail da się uruchomić NAPRAWDĘ rozproszony?

Po zakończeniu podstawowej weryfikacji zadano ważniejsze pytanie: czy Sail w trybie „local-cluster” (który brzmi jak „klaster”) rzeczywiście uruchamia osobne procesy komputerowe, czy tylko to udaje?

Jak to sprawdzono

Zrobiono „zdjęcie” działającego systemu w trakcie liczenia — listę wszystkich działających procesów oraz listę wszystkich otwartych „gniazd sieciowych” (portów), na których coś nasłuchuje. Wynik: **wszystkie cztery role** — „dyrygent” (driver), dwóch „robotników” (worker 1, worker 2) i serwer komunikacyjny — należały do **jednego i tego samego procesu** na komputerze (jeden numer PID). To trochę jak firma, która ma cztery szyldy na drzwiach różnych biur, ale w środku wszystkie „biura” to w rzeczywistości jedna i ta sama osoba, przebijająca się w różne kostiumy i biegająca między pokojami.

Dodatkowym, przypadkowym potwierdzeniem było to, że próba użycia „kłódki” (threading.Lock) do zablokowania jednoczesnego dostępu dwóch workerów do tych samych danych **nie zadziałała** — dostano błąd, że kłódki nie da się „spakować i wysłać” (zserializować). To dowód, że mimo bycia jednym procesem, workery faktycznie przesyłają między sobą kod jako gotowe, samodzielne paczki (tak jakby naprawdę były osobnymi komputerami) — więc czegoś tak „fizycznego” jak kłódka nie da się między nimi przekazać. Zamiast kłódki użyto innego triku: próbuj ponownie po chwili, jeśli akurat ktoś inny w tym momencie korzysta z tych samych danych.

Wniosek

Prawdziwa, wieloprocusowa/wielokomputerowa dystrybucja w Sailu wymaga innego trybu — **Kubernetes** (KubernetesCluster). Bez Kubernetesa, „local-cluster” to tylko symulacja wyglądająca jak klaster, ale fizycznie działająca w jednym procesie.

Co to jest Kubernetes i dlaczego to komplikuje sprawę

Kubernetes to system do zarządzania wieloma „kontenerami” (odizolowanymi, przenośnymi paczkami z programem i wszystkim, czego potrzebuje do działania) rozsianymi po wielu maszynach — sam decyduje, gdzie co uruchomić, restartuje, jeśli coś padnie, itd. To potężne narzędzie, ale też ciężkie — normalnie uruchamia się je na serwerach z dużą ilością pamięci.

Skoro maszyna testowa ma tylko **3,5GB RAM łącznie** (bardzo mało jak na tego typu zadania — dla porównania, przeciętny nowy laptop ma 16GB), a raz już zdarzyło się, że zwykła kompilacja programu zawiesiła cały komputer, postanowiono najpierw sprawdzić na chłodno, ile realnie pamięci taki eksperyment by zjadł, zanim cokolwiek zainstalowano.

Znaleziono lżejszą wersję Kubernetesa — k3s — reklamowaną jako potrzebującą tylko 2GB RAM minimum. To wciąż bardzo blisko granicy całej dostępnej pamięci na tej maszynie, więc zaplanowano dodatkowe zabezpieczenie: **twardy limit pamięci** dla samego procesu k3s, żeby jeśli przekroczy on bezpieczną granicę, został szybko i czysto zatrzymany, zamiast powoli dusić cały system (co jest dokładnie tym, co się stało przy poprzednim zawieszeniu — komputer nie „umarł” od razu, tylko stawał się coraz wolniejszy, aż przestał odpowiadać).

Co to jest taki „limit pamięci” (cgroup)

Wyobraź sobie, że każdemu programowi można przypisać własną, oddzielną „kopertę” z ograniczoną ilością pieniędzy (pamięci) do wydania. Jeśli program spróbuje wydać więcej niż ma w kopercie, zostaje natychmiast zatrzymany — reszta budżetu domowego (systemu) zostaje nietknięta. Mechanizm w Linuksie, który to umożliwia, nazywa się **cgroup** (*control group*). Dodatkowo poproszono system, żeby dla tej konkretnej „koperty” w ogóle nie pozwalał na sięganie po dodatkowy, awaryjny fundusz (**swap**, czyli miejsce na dysku używane jako zapasowa, wolniejsza pamięć) — żeby zamiast powolnego wpadania w długi, program od razu dostał twarde „nie” i się zatrzymał.

Przy okazji zwiększono też ogólny, awaryjny fundusz (swap) dla reszty systemu z 1GB do 3GB, oraz zmieniono ustawienie mówiące systemowi, jak chętnie ma sięgać po ten fundusz zamiast czyścić pamięć na bieżąco (swappiness) — z wartości „dość chętnie” na „raczej niechętnie”, żeby ewentualne problemy kończyły się szybko i czysto, a nie długim dogorywaniem systemu.

Co poszło nie tak

Instalacja k3s przebiegła bez problemu. Problem pojawił się przy próbie odpalenia go w typowy sposób (przez `systemd` — standardowy „zarządca usług” w Linuksie, coś w rodzaju recepcjonisty, który uruchamia i pilnuje programów działających w tle). Okazało się, że to konkretne środowisko (WSL — sposób uruchamiania Linuksa wewnątrz Windowsa) **w ogóle nie ma tego recepcjonisty włączonego** — wcześniejsze sprawdzenie tego faktu było błędne (sprawdzono tylko, czy istnieje odpowiednie narzędzie, a nie czy faktycznie działa jako „prawdziwy” zarządca).

Naprawienie tego na stałe wymagałoby pełnego restartu całego środowiska Linux w Windowsie — co ubiłoby też bieżącą sesję pracy, więc uznano to za zbyt ryzykowne. Zamiast tego spróbowano odpalić k3s „z ręki”, bezpośrednio umieszczając go w przygotowanej „kopercie” (cgroup) — ale próba zapisania limitu do tej koperty została odrzucona przez system z komunikatem „odmowa dostępu”, **nawet dla administratora**. Innymi słowy: mechanizm bezpieczeństwa, na którym opierał się cały plan, okazał się niedostępny w tym konkretnym środowisku.

Co się stało dalej

k3s wystartował — ale **bez żadnego limitu**. W ciągu około minuty, jeszcze zanim zdążył w pełni się uruchomić (był w trakcie instalowania swoich domyślnych, dodatkowych

komponentów), zjadł niemal całą dostępną pamięć komputera (z 2,1GB dostępnego zapasu do 171MB). To dokładnie ten scenariusz, którego cała ta ostrożność miała nie dopuścić.

Zareagowano natychmiast: zatrzymano proces oficjalnym, wbudowanym w k3s poleceniem do czystego wyłączenia wszystkiego, co uruchomił (włącznie z jego procesami pomocniczymi). Pamięć od razu wróciła do normy, komputer nie ucierpiał. Na koniec odinstalowano k3s całkowicie, zostawiając system czysty.

Co z tego wynika

To jest **wynik negatywny, ale bardzo wartościowy i rzetelnie sprawdzony**: nie „w teorii pewnie by nie działało”, tylko „faktycznie spróbowaliśmy, ostrożnie, z zabezpieczeniami, i realnie zabrakło zasobów zanim mechanizm zabezpieczający zdążył pomóc”. To ważny wniosek porównawczy do pracy: na tej samej maszynie Ballista osiągnęła pełną, prawdziwą dystrybucję (Rozdział 2), a Sail — nie, z jasno wskazanym i udokumentowanym powodem: jego jedyna lokalnie dostępna, naprawdę-wieloprocusowa ścieżka wymaga Kubernetesa, a ten akurat na tym sprzęcie się nie mieści.

Rozdział 4 — Co to wszystko znaczy dla całej pracy magisterskiej

Promotor postawił jasne wymagania: mechanizm dodawania własnych funkcji do silnika rozproszonego bez jego modyfikowania (forkowania), i to dla **obu** silników — Ballista i Sail. Oba te warunki zostały spełnione dla wszystkich pięciu podstawowych operacji genomicznych (overlap, merge, nearest, coverage, subtract), a dla najważniejszej z nich (overlap) osiągnięto dodatkowo pełną dystrybucję z zachowaniem najszybszego znanego algorytmu (COITrees) — i to tylko w Ballistrze, co samo w sobie jest ciekawym wynikiem porównawczym (Ballista, będąca bezpośrednią nakładką na DataFusion, ma dziś dojrzalszy mechanizm rozszerzeń niż Sail, który dopiero projektuje swój odpowiednik).

Właściwe „wpisanie” tego mechanizmu na stałe do samej biblioteki polars-bio (żeby użytkownik mógł po prostu napisać `pb.overlap(..., engine="ballista")`) zostało świadomie odłożone — bo to decyzja, którą powinien podjąć promotor jako współautor tej biblioteki, a nie coś, co powinno zostać „przy okazji” wpisane na stałe bez jego zgody.

Do zrobienia zostają przede wszystkim: prawdziwe pomiary wydajności na większych danych i większej liczbie komputerów (najpewniej w chmurze, bo lokalna maszyna, jak pokazał ten rozdział, ma zbyt mało zasobów na cięższe eksperymenty), oraz rozmowa z promotorem podsumowująca te wyniki i ustalająca dalsze priorytety.